AI Resume Screening

Interview Defence Guide

Every resume line explained, evidenced and defensible — with diagrams, plain-English NLP terminology and an honest answer to every awkward question. Verified by running the code.

Python · Streamlit · Sentence Transformers · Cosine Similarity · PyMuPDF

What is inside

- **Resume check.** Two claims on this project's resume lines are not supported by the code (experience / education extraction, and TF-IDF). Chapter 1 shows them and gives corrected wording.
- **Resume-first structure.** Chapter 4 takes your three bullets one by one: plain-English meaning, proof in the repo, terms, follow-ups.
- **Verified by running the code.** I ran the skill matcher, the parser and the scoring maths on edge cases, so the weaknesses in this guide are demonstrated, not guessed.
- **9 diagrams**, a 45-term glossary, 50+ questions with answers, trap questions and a one-page cheat sheet.

Prepared for **Snehil Rai** · generated 15 Oct 2026 from `AI-Resume-Screening-System-main.zip` and your resume.

Contents

01 Read this first: your resume vs your repository . . . 3
02 The project in 30 seconds, 90 seconds and one page . . . 5
03 Architecture and request flow . . . 7
04 Your resume lines, one by one . . . 10
05 NLP deep dive: from text to a similarity score . . . 14
06 Skill matching and hybrid scoring deep dive . . . 17
07 The app, parsing and engineering practice . . . 20
08 Evaluation, limitations and the improvement roadmap . . . 23
09 Glossary: terminology in plain English . . . 25
10 Question bank: more than 50 questions with answers . . . 27
11 Trap questions and honest limitations . . . 32
12 One-page cheat sheet . . . 33
13 Appendix A: run it and demo it in two minutes . . . 34
14 Appendix B: where things are in the repository . . . 35

How to practise with this guide

- **Day 1:** read Chapter 1 and fix your resume and README. Read Chapters 2 and 3 aloud.
- **Day 2:** read Chapter 4 (resume lines), Chapter 5 (NLP) and Chapter 6 (scoring). Say each glossary definition out loud.
- **Day 3:** run the app once (Appendix A), then work through the question bank without looking.
- **Interview morning:** read the cheat sheet (Chapter 12) and the honest-limitations statement.

> **IN PLAIN ENGLISH**
> This is a small project (336 lines of Python in four files). That is an advantage: an interviewer can read all of it in ten minutes, so you must be able to explain **every line**. Learn, for each idea, **the problem it solves, how data flows through it, where it lives in the code, the trade-off and the limitation**.

01 Read this first: your resume vs your repository

I compared each phrase of your three bullets and your skills line with the code, and I ran the code to check behaviour. This project has **no numeric accuracy claims** on the resume, which is good, so nothing is quantitatively wrong. But **two claims describe features that do not exist**, and one needs careful wording.

Resume says	Repository shows	Verdict	Use instead
parses PDF/DOCX files	src/parser.py: PyMuPDF for PDF, python-docx for DOCX. Caveats I tested: DOCX tables, headers and footers are not read; scanned PDFs return nothing.	✓ **matches**	Keep, but know the caveats
extracts structured fields such as skills, **experience, and education**	Only **skills** are extracted, by matching a fixed list of 92 keywords. There is no code that reads years of experience, job titles, degrees or education sections.	✗ **not in code**	"extracts skills against a 92-keyword vocabulary"
ranks candidates against a job description	app.py sorts results by final score, highest first.	✓ **matches**	Keep
Sentence Transformers and cosine similarity … contextual meaning beyond exact keyword matching	all-MiniLM-L6-v2 embeddings + scikit-learn cosine_similarity (embedder.py). "Beyond keyword matching" is the design intent; the repo has no evaluation showing it actually ranks better.	⚠ **careful**	Keep; say "designed to capture", never "proved"
hybrid scoring: 70% semantic + 30% keyword skill matching	app.py: `0.7 * semantic_score + 0.3 * skill_score`	✓ **matches**	Keep
presented strengths, skill gaps, fit scores, CSV export	Strengths and gaps (max 5 each), three scores, Strong/Moderate/Not Fit label, an explanation sentence, and a CSV download.	✓ **matches**	Keep
Skills line: "NLP, Sentence Transformers, **TF-IDF**, Cosine Similarity"	No TF-IDF anywhere in the repo (no TfidfVectorizer). scikit-learn is used only for cosine_similarity.	✗ **not in code**	Remove TF-IDF, or add a TF-IDF baseline (Chapter 8) so it becomes true

FIX BEFORE YOUR INTERVIEW

1. Change "extracts structured fields such as skills, experience, and education" to "extracts skills against a 92-keyword vocabulary", **or** implement experience / education extraction before you keep the sentence (a realistic 1–2 day addition; see Chapter 8).
2. Remove "TF-IDF" from the skills line unless you have genuinely used it elsewhere. Interviewers love to ask "where did you use TF-IDF?".
3. README: replace the placeholder clone URL (`github.com/your-username/ai-resume-screening`) and the folder name in the structure diagram with your real repository name; add a screenshot.

Corrected resume bullets (copy-paste ready)

SUGGESTED BULLETS

• Built a Streamlit resume-screening app that parses PDF and DOCX resumes (PyMuPDF, python-docx), extracts skills against a 92-keyword vocabulary, and ranks candidates against a pasted job description.

• Embedded resumes and job descriptions with the all-MiniLM-L6-v2 Sentence Transformer and scored them by cosine similarity to capture meaning beyond exact keyword matches.

• Implemented hybrid scoring (70% semantic similarity, 30% keyword skill coverage) with Strong / Moderate / Not Fit labels; displayed strengths, skill gaps and fit scores per candidate, with CSV export.

What I verified, and what I could not

Checked by me (by running the code)	Not checked by me
Read all four source files, the README and requirements. Ran the skill matcher on 25+ test sentences (Chapter 6). Created a text PDF, a scanned-style PDF, a DOCX with a table and header, a .doc and a corrupt PDF and ran the real parser on them. Ran the real scoring maths and the 65 / 40 thresholds, using a stub in place of the embedding model.	The real all-MiniLM-L6-v2 outputs: the sandbox cannot download from Hugging Face. Statements about the model (384 dimensions, 256-token limit, training data) come from its published model card. Launching the Streamlit UI. Ranking quality: the repo contains no labelled data, tests or evaluation, so there is nothing to measure against. Anything about deployment or real users.

02 The project in 30 seconds, 90 seconds and one page

30-second pitch

SAY IT LIKE THIS

"AI Resume Screening is a Streamlit app where a recruiter pastes a job description and uploads many PDF or DOCX resumes. For each resume it computes two scores: a semantic score, using a pretrained Sentence Transformer and cosine similarity, and a keyword skill-coverage score. It blends them 70 / 30 into a fit score, ranks the candidates, and shows strengths, skill gaps and a Strong, Moderate or Not Fit label, with CSV export. It is decision support for a human recruiter, not an automatic hiring decision."

90-second pitch

SAY IT LIKE THIS

"Recruiters face hundreds of resumes per role, and keyword search misses people who describe the same skill in different words. So I built a hybrid screener. The **semantic half** turns each resume and the job description into a 384-number embedding with the all-MiniLM-L6-v2 model and takes the cosine similarity, which captures meaning rather than exact words. The **keyword half** checks how many of the skills named in the job description appear in the resume, which is easy to explain and shows exactly which skills are missing. I combine them 70 / 30, map the result to Strong, Moderate or Not Fit, and show strengths, gaps and a plain-English explanation, plus a CSV export."

"It is intentionally simple and explainable. The main limitations are that skills come from a fixed list of 92 keywords, the weights and thresholds are design choices that I have not tuned against labelled data, and the embedding model only reads the first few hundred words of each document. The next step would be a small labelled evaluation."

The one-page picture

	Semantic half	Keyword half
Question it answers	"Does this resume talk about the same things as the job description?"	"How many of the skills the JD names does the resume mention?"
Technique	Sentence-Transformer embeddings (all-MiniLM-L6-v2) + cosine similarity, rescaled from 0.20–0.95 to 0–100	92-keyword vocabulary, regex word-boundary matching, set intersection ÷ JD skills × 100
Strength	Handles paraphrase and context ("built ETL jobs" ≈ "data pipelines")	Transparent: you can see exactly which skills matched or are missing
Weakness	Opaque; general-purpose model; reads only the first ~256 word-pieces; score scale is a heuristic	Brittle: misses synonyms and "C++", falsely matches "excel", "go", "express"
Weight / output	70% of final score	30% of final score; feeds strengths and gaps
Where	`src/embedder.py`	`src/analyzer.py`

IN PLAIN ENGLISH

Why hybrid? Each half covers the other's blind spot. Embeddings are good at meaning but cannot tell you *why* a score is high; keywords can explain themselves but miss paraphrases. A reviewer can trust the strengths / gaps lists because they are literal matches. Be ready to say this in one breath: *"semantic for coverage of meaning, keywords for explainability."*

Why this project suits a software / data / AI role

SAY IT LIKE THIS

“It is a small but complete pipeline: ingest unstructured documents (PDF, DOCX), clean and normalise text, turn it into features (embeddings and skill sets), score and rank, and present the result with an explanation and an export. I made deliberate design decisions, such as caching the model, validating inputs, keeping scoring logic in separate modules, and I can tell you where it breaks.”

DON'T SAY

- “It extracts experience and education.” → It extracts **skills only**.
- “It is X% accurate.” → There is no labelled evaluation; any accuracy number would be invented.
- “It removes bias.” → It does not measure or remove bias; it needs a human in the loop.
- “It uses TF-IDF.” → It does not.
- “It is production-ready / used by recruiters.” → It is a prototype with no tests, authentication or deployment.

03 Architecture and request flow

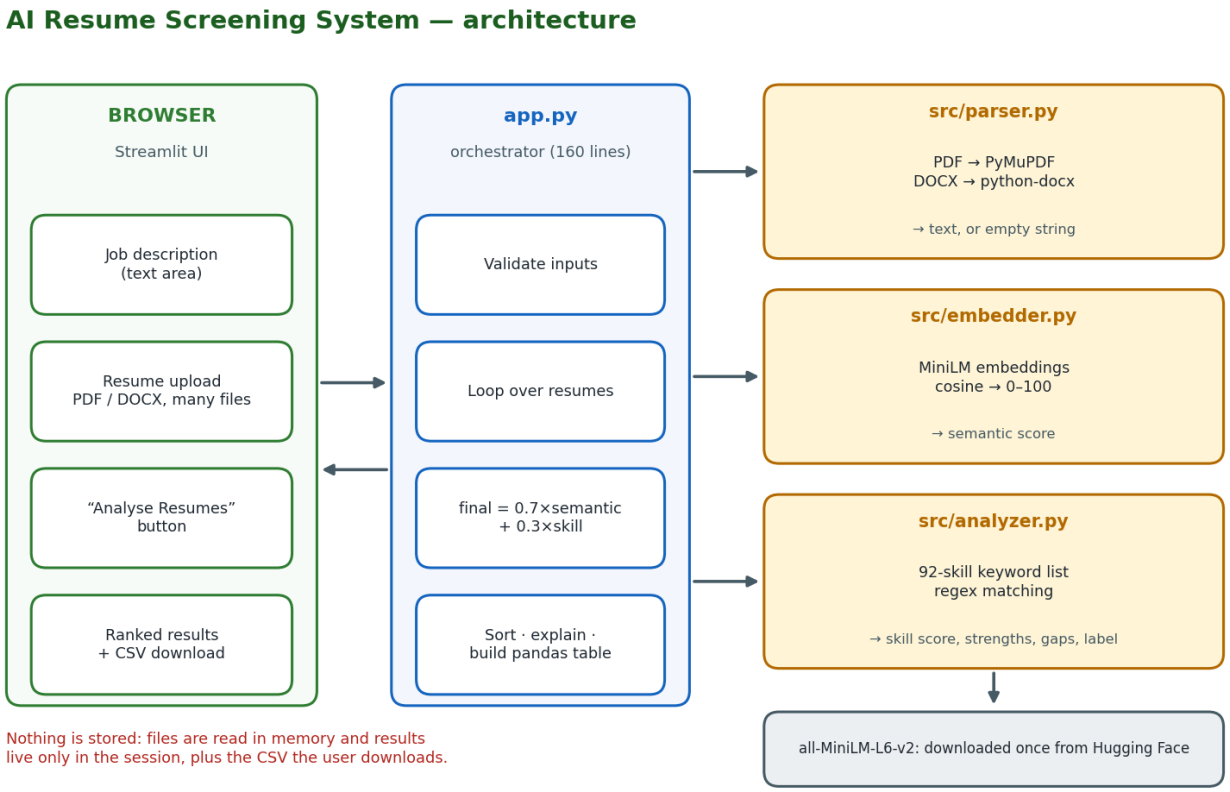
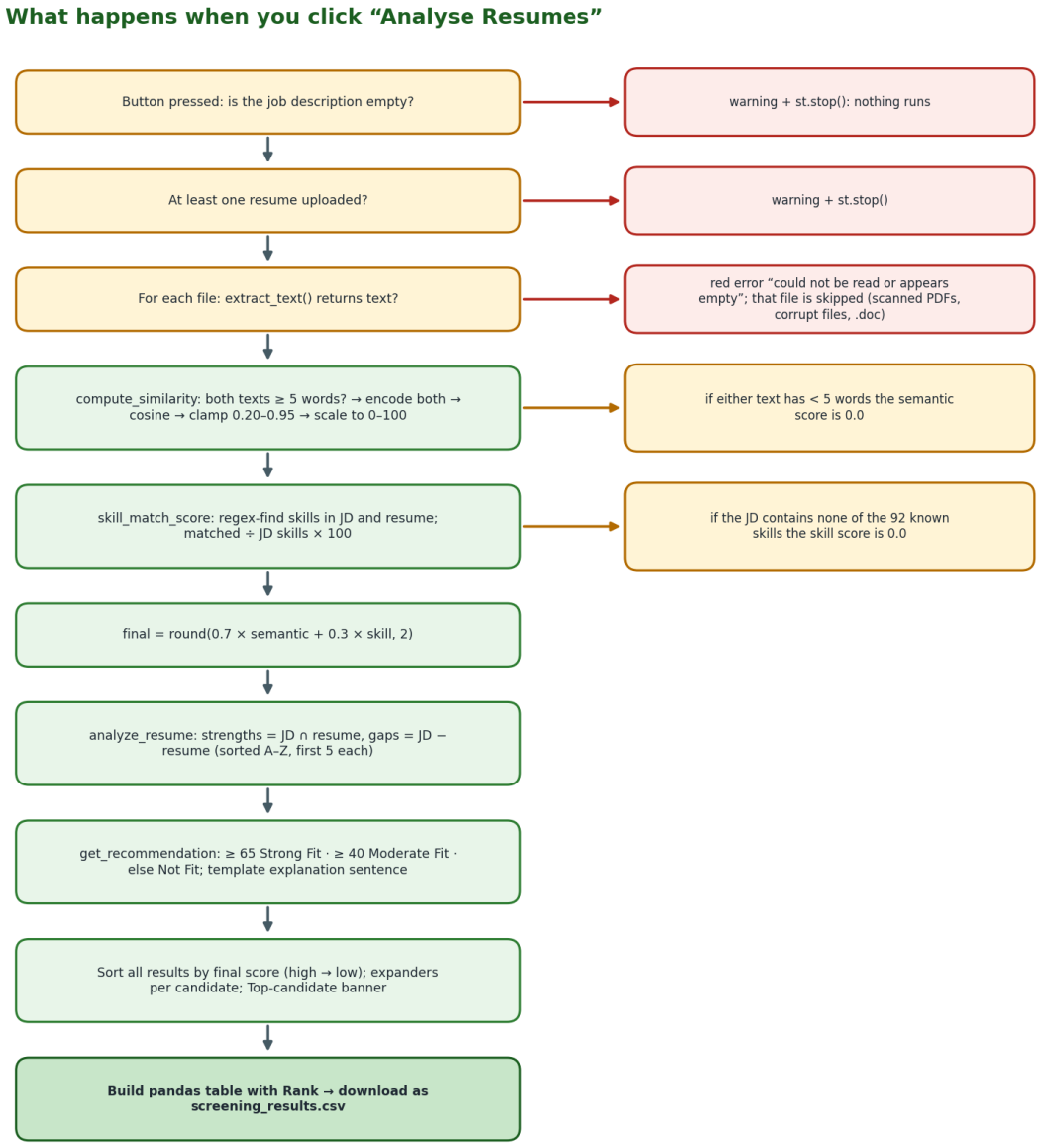


Figure 1. One Streamlit page, one orchestrating script and three small logic modules. Nothing is written to disk or a database.

File	Lines	Responsibility
`app.py`	160	Streamlit page: inputs, validation, loop over resumes, hybrid score, sorting, results display, CSV export
`src/parser.py`	37	extract_text(file): PDF (PyMuPDF) or DOCX (python-docx) → plain string, or empty string on any problem
`src/embedder.py`	45	Loads the model once (st.cache_resource), validates text, encodes, cosine similarity, normalises to 0–100
`src/analyzer.py`	94	92-skill vocabulary, regex skill extraction, strengths and gaps, skill score, Strong / Moderate / Not Fit label
`requirements.txt`	6	streamlit, sentence-transformers, scikit-learn, pymupdf, python-docx, pandas (lower bounds only, no pins)

What happens when you click “Analyse Resumes”



If every file fails, the app shows “No resumes could be processed” and stops. Amber = decision/edge case, green = step, red = user-facing error.

Figure 2. Ten steps. Every exit is a visible message; nothing fails silently except that parse errors are reported with one generic message.

What the user can see

Situation	UI behaviour
Job description empty	Warning “Please paste a Job Description”, processing stops
No file uploaded	Warning “Please upload at least one resume”, processing stops
One file unreadable	Red error naming that file; the other files continue
All files unreadable	“No resumes could be processed” and stop
Normal	Top-candidate banner; one expander per candidate (rank, colour icon, final score, label) with Semantic, Skill and Final metrics, a coverage bar, strengths, gaps and an insight sentence; CSV download button

Design choices you can claim (all visible in the code)

- **Separation of concerns.** Parsing, embedding and skill logic are separate modules with small functions; the page only orchestrates.
- **Load the model once.** `@st.cache_resource` keeps one model instance across Streamlit's frequent script reruns.

- **Explainable output.** Matched and missing skills are literal, and the explanation sentence is built from them.
- **Input checks.** Empty JD, no files, unreadable file, texts shorter than 5 words are all handled explicitly.
- **Privacy by simplicity.** Files are read in memory and nothing is stored; the only output file is the CSV the user downloads.

04 Your resume lines, one by one

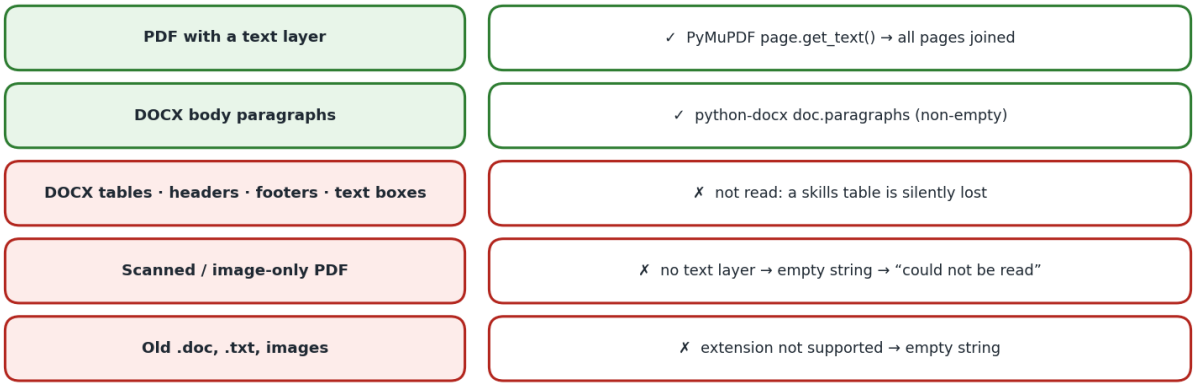
Each section takes one bullet (in its **corrected** form from Chapter 1) and gives a phrase-by-phrase breakdown, proof in the repo, terminology and likely follow-ups.

Bullet 1 · Parsing, skill extraction and ranking

> **THE LINE**
> Built a Streamlit resume-screening app that parses PDF and DOCX resumes (PyMuPDF, python-docx), extracts skills against a 92-keyword vocabulary, and ranks candidates against a pasted job description.

Phrase	What it means	Proof
Streamlit app	Streamlit turns a Python script into a web page: widgets (text area, file uploader, button) and output calls (metric, expander, download button). The script re-runs top to bottom on every interaction.	`app.py`
parses PDF	PyMuPDF opens the uploaded bytes and joins each page's text layer. Scanned PDFs (pictures of text) have no text layer, so they return nothing.	`parser._extract_pdf`
parses DOCX	python-docx reads the body paragraphs. Text inside tables, headers, footers and text boxes is **not** read (I tested a skills table: it was lost).	`parser._extract_docx`
extracts skills against a 92-keyword vocabulary	The text is lower-cased, whitespace collapsed, and each of 92 keywords is searched with a word-boundary regular expression. The result is a set of matched keywords.	`analyzer.SKILLS`, `extract_skills()`
ranks candidates	Results are sorted by final score, descending; the best is shown first and in a banner.	`results.sort(...)`
pasted job description	The JD is a plain text area; its skills are extracted with the same function.	`st.text_area`

What parser.py can and cannot read (tested with generated files)



Any exception while parsing is swallowed (except Exception → ""), so the user sees one generic error whatever the cause.

Figure 3. Parser coverage as tested with generated files. The silent loss of table content is the most important caveat.

Questions this bullet invites

Q. Why only 92 keywords? Why not extract skills automatically?

"I wanted transparent, deterministic matching for the explainable half of the score, and a curated list is easy to audit. The cost is that anything outside the list is invisible. The natural next step is to learn the vocabulary from job descriptions, or use named-entity recognition or an LLM, and to add synonyms."

Q. Does it extract experience and education?

"No, only skills. Years of experience and education would need section detection and date parsing or NER, which I have listed as a next step." (Correct your resume so this question never catches you out.)

Q. What if the resume is a scanned PDF or has a two-column layout?

"A scanned PDF has no text layer, so the app reports that the file could not be read; adding OCR with something like Tesseract would fix that. Two-column layouts can interleave text from both columns when extracted, which hurts both scores; layout-aware parsing would be needed."

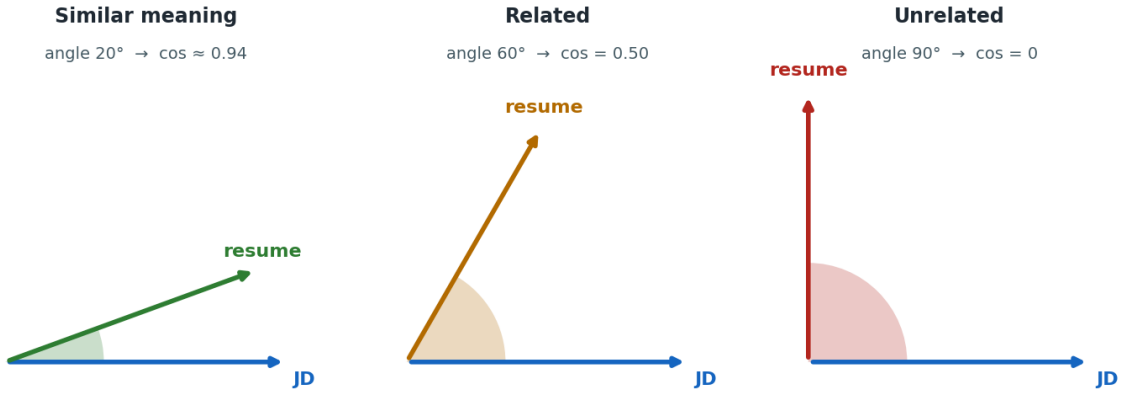
Q. How does ranking work with ties or one file?

"Python's sort is stable, so equal scores keep upload order. With one file it just ranks a single candidate and the banner still shows it."

Bullet 2 · Sentence Transformers and cosine similarity

> **THE LINE**
> Embedded resumes and job descriptions with the all-MiniLM-L6-v2 Sentence Transformer and scored them by cosine similarity to capture meaning beyond exact keyword matches.

Phrase	What it means	Proof
Sentence Transformer	A BERT-style neural network trained so that whole sentences or passages with similar meaning end up with similar vectors (embeddings).	`SentenceTransformer("all-MiniLM-L6-v2")`
all-MiniLM-L6-v2	A small, fast model: 6 transformer layers, 384-dimensional output, trained on over a billion sentence pairs; inputs longer than 256 word-pieces are truncated (from the model's published card; I could not download it here).	`embedder.load_model`
embedding	A list of 384 numbers representing the meaning of a text. Similar meanings → vectors pointing in similar directions.	`model.encode(..., convert_to_numpy=True)`
cosine similarity	The cosine of the angle between two vectors: 1 means the same direction, 0 unrelated. It ignores vector length.	`sklearn.metrics.pairwise.cosine_similarity`
normalised to 0–100	The raw cosine is clamped to [0.20, 0.95] and linearly stretched, so the result reads like a percentage.	`_normalize_score()`
beyond exact keyword matches	Embeddings place "built ETL jobs" near "data pipeline development" even though the words differ.	design intent, not measured



Each text becomes a vector (384 numbers for MiniLM; drawn here in 2-D).
Cosine similarity = cos(angle): 1 = same direction, 0 = unrelated. Illustration, not measured data.

Figure 4. Cosine similarity as the angle between two vectors (2-D drawing of a 384-D idea).

Questions this bullet invites

Q. How do you know the semantic score is meaningful?

"I do not have a labelled evaluation, so I cannot give an accuracy. I rely on the fact that the model is a widely used, pretrained general-purpose sentence encoder, and I use cosine similarity as a relative ranking signal rather than an absolute truth. A proper check would compare it with recruiter rankings on a small labelled set."

Q. Why cosine similarity and not Euclidean distance?

"Cosine compares direction (meaning) and ignores vector length, which is what sentence-embedding models are trained for. It is also bounded and easy to interpret."

Q. Why that model?

"It is small and fast on a CPU, needs no GPU, downloads once, and gives good general-purpose sentence similarity. A larger model such as all-mpnet-base-v2 may be more accurate and reads longer inputs, at the cost of speed; I did not benchmark alternatives."

Q. Where did 0.20 and 0.95 come from?

"The code calls them empirical bounds: raw cosines for real resume–JD pairs sit in a middle band, so I stretch that band to 0–100 so scores read naturally. They are a presentation heuristic, not statistically derived, and I would recalibrate them on labelled data." (Say it honestly. The repository does not show how they were chosen.)

Bullet 3 · Hybrid scoring and the output

THE LINE

Implemented hybrid scoring (70% semantic similarity, 30% keyword skill coverage) with Strong / Moderate / Not Fit labels; displayed strengths, skill gaps and fit scores per candidate, with CSV export.

Phrase	What it means	Proof
hybrid scoring	Combine two different signals into one number: `0.7 × semantic + 0.3 × skill`.	`app.py` line "Hybrid score"
70% semantic	Semantic score (0–100) is the normalised cosine.	`compute_similarity()`
30% keyword skill coverage	Share of the JD's recognised skills that the resume also mentions, ×100.	`skill_match_score()`
Strong / Moderate / Not Fit	Final ≥ 65 is Strong Fit; ≥ 40 is Moderate Fit; otherwise Not Fit.	`get_recommendation()`
strengths and gaps	Strengths = skills in both JD and resume; gaps = JD skills missing from the resume. Each list is sorted A–Z and cut to 5.	`analyze_resume()`
fit scores	Semantic, Skill and Final are shown as metrics per candidate.	`c1.metric(...)`
CSV export	A pandas DataFrame with Rank, Name (the file name), three scores, strengths, gaps, label and explanation, downloaded as screening_results.csv.	`st.download_button`

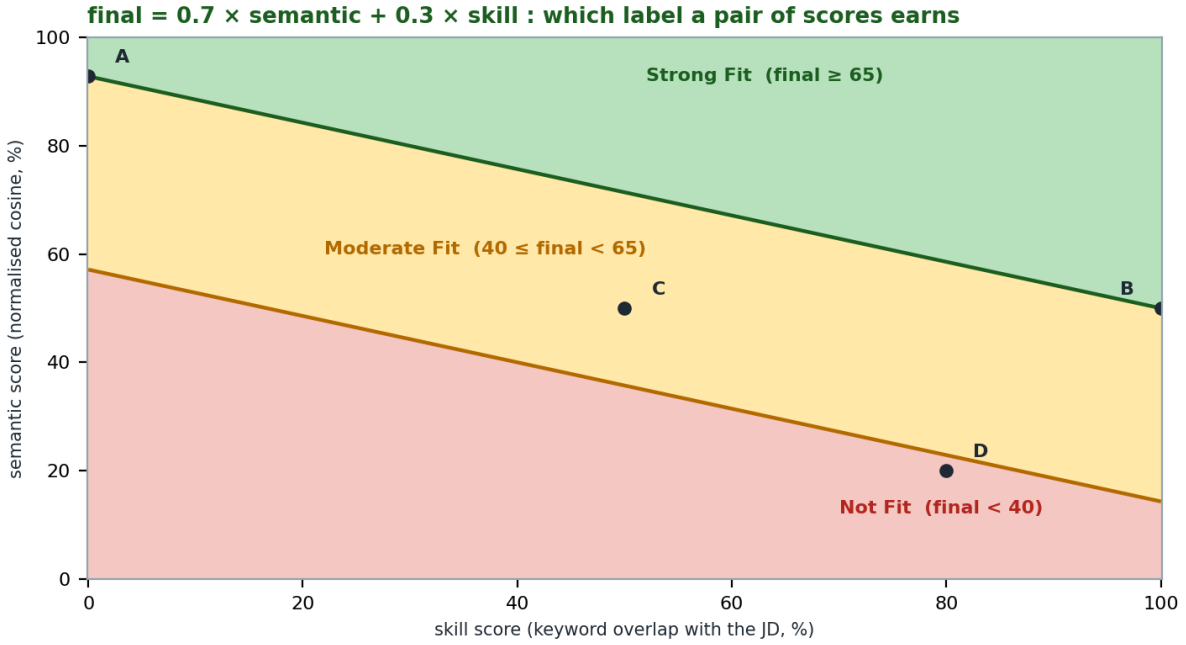


Figure 5. The two scores combine along straight lines. A: JD with no known skills; B: perfect skill match; C: middling both (final 50); D: weak semantics, strong keywords (final 38).

Questions this bullet invites

Q. Why 70 / 30?

"It is a design choice, not a tuned value: I wanted meaning to dominate because keywords are brittle, while keeping a substantial, explainable keyword component. I did not have labelled data to tune it, so the right next step is to try other weights and compare rankings with recruiter judgements."

Q. What does a final score of 65 mean?

"Only that this resume scored 65 on my blended scale. It is not a probability of being a good hire. The 65 and 40 cut-offs are my labels; the code comment says they follow the assessment specification."

Q. Why show a recommendation and an explanation?

"So a recruiter can see why: the explanation lists matched and missing skills and the label. It keeps the human in control instead of presenting an unexplained number."

Q. What do the 'Name' values in the CSV contain?

"The uploaded file name, not a name extracted from the resume. Names are not parsed."

05 NLP deep dive: from text to a similarity score

Embeddings in plain English

IN PLAIN ENGLISH

The idea. Computers cannot compare meanings directly, so we convert text to a list of numbers (a *vector*) such that texts with similar meaning get similar numbers. “Similar” is then just geometry: the angle between two arrows.

Word vs sentence embeddings. Word2Vec gives one vector per word. Sentence-Transformers (SBERT) fine-tune BERT so that a *whole sentence or passage* maps to one useful vector, which is what you need to compare a resume to a job description.

What “MiniLM-L6” means. A distilled (compressed) transformer with 6 layers that keeps most of a bigger model's quality at a fraction of the cost; “v2” is the sentence-embedding fine-tune.

The model facts to remember

Fact	Value	Note
Model name	`all-MiniLM-L6-v2`	Loaded once with st.cache_resource; downloaded on first run (README note)
Output size	384 numbers per text	Published model card
Maximum input	256 word-pieces	Longer text is **truncated**. 256 word-pieces is roughly 150–200 English words
Training	Over 1 billion sentence pairs, contrastive learning	Published model card: pairs of related texts are pulled together
Similarity	Cosine, via scikit-learn	Raw value roughly 0–1 for typical text pairs

These come from the model's public documentation; I could not download the model in my sandbox, so I did not re-measure them.

Cosine similarity and the 0–100 rescale

For vectors **a** and **b**: cosine = (a · b) ÷ (|a| × |b|). The app then maps it with `score = (clamp(cos, 0.20, 0.95) − 0.20) ÷ 0.75 × 100`.

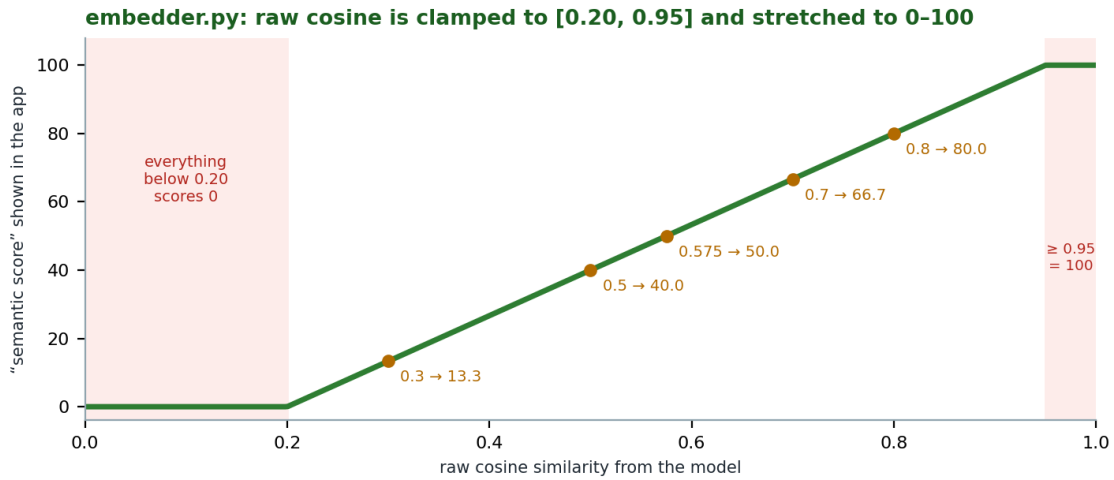


Figure 6. Verified by running the real function with a stub model: 0.3 → 13.33, 0.5 → 40.0, 0.575 → 50.0, 0.7 → 66.67, 0.8 → 80.0.

WATCH OUT

What the rescale does and does not do. It preserves the *order* of candidates (it is monotonic) so ranking is unaffected by the exact bounds. It does change how the semantic score blends with the skill score and where the 65 / 40 labels fall, so the bounds quietly shape the labels.

Honest framing: it is a readability heuristic. A cosine of 0.575 shows as “50% semantic match”, but that is not a probability of anything.

Consequences of embedding a whole document as one vector

Issue	Why it matters here	Fix you could propose
Truncation at 256 word-pieces	A typical resume is 400–900 words. Only roughly the first 150–200 words (the header, summary and first skills) reach the model, so later projects and experience may not influence the semantic score at all. The same applies to long job descriptions.	Split into chunks (sliding windows or sections), embed each, and aggregate (mean or best match); or use a model with a longer window
One vector for everything	Different sections (skills, experience, education) are blended into one meaning.	Compare JD requirements with individual resume sentences; use the best matches as evidence
General-purpose model	It does not know that “Airflow” and “dbt” are data-engineering signals as well as a trained domain model would (README limitation).	Fine-tune on resume–JD pairs or use a domain model
Re-encoding the JD per resume	compute_similarity encodes the JD again for every resume: 2N encodes instead of N + 1.	Encode the JD once and the resumes in a batch
Score ≠ qualification	Two resumes with the same wording can differ greatly in seniority.	Add experience and seniority features

TF-IDF vs embeddings (it is on your skills line, so know it)

WATCH OUT

TF-IDF is **not used in this project**. Know it well enough to compare it, and remove it from your resume unless you have used it elsewhere.

	TF-IDF	Sentence embeddings (this project)
Idea	Weight each word by how often it appears in the document (term frequency) times how rare it is across documents (inverse document frequency): rare, distinctive words count more.	A neural network maps the whole text to a dense vector of meaning.
Vector	Sparse: one dimension per vocabulary word (thousands, mostly zeros)	Dense: 384 numbers
Matches	Only shared words. “ETL” and “data pipeline” are unrelated.	Related meanings even with different words
Explainability	High: you can list the shared words	Low: a number with no reasons
Needs training	Only a corpus to count in	A pretrained neural model
Typical use	Keyword relevance baseline	Semantic search, matching
In scikit-learn	`TfidfVectorizer` then `cosine_similarity`	`SentenceTransformer.encode` then `cosine_similarity`

A good upgrade: add a TF-IDF cosine score as a **third** signal or as a baseline in your evaluation. It takes about 15 lines and makes the resume claim true.

Where this sits among alternatives

Approach	In one line	Trade-off
Keyword / boolean search	Does the resume contain the words?	Transparent but brittle (this project's skill half)
TF-IDF + cosine	Weighted word overlap	Simple baseline; no semantics
Bi-encoder embeddings (this project)	Embed each text separately, compare vectors	Fast, scalable; moderate accuracy
Cross-encoder re-ranker	A model reads the JD and the resume together and scores the pair	More accurate, much slower; usually applied to a shortlist
LLM-based screening	Prompt a large language model to evaluate the resume	Flexible and can explain; costly, inconsistent, and higher bias and privacy risk
NER / information extraction	Pull out skills, titles, dates, degrees as structured fields	Enables experience / education features; needs a model or rules

06 Skill matching and hybrid scoring deep dive

How the keyword half works

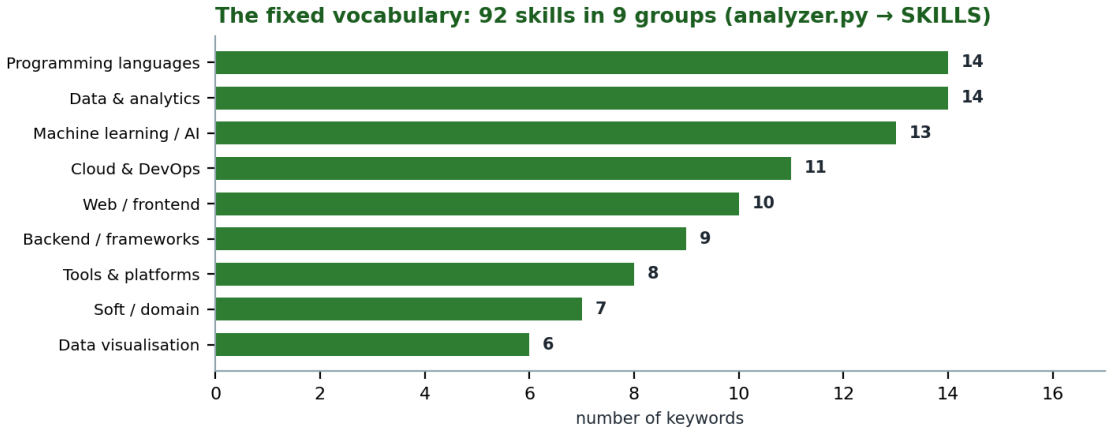
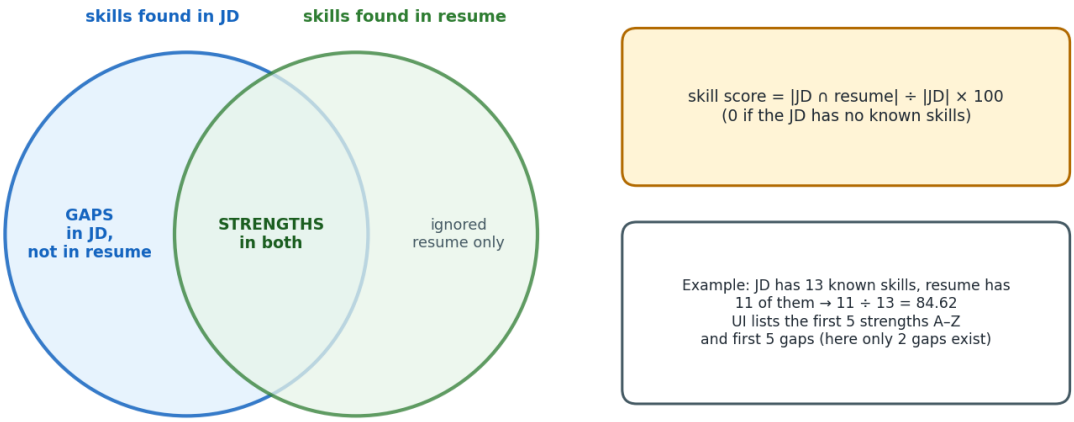


Figure 7. The vocabulary skews technical: programming, data, ML and cloud make up 52 of 92 keywords; only 7 are soft or domain terms.

How strengths, gaps and the skill score come from two sets



Extra skills on the resume earn nothing: the score only measures how much of the JD's vocabulary the resume covers.

Figure 8. Strengths and gaps are plain set operations on the skills found in the JD and the resume.

IN PLAIN ENGLISH

Regular expression with word boundaries. For each keyword the code searches `\bkeyword\b`. The **\b** means “edge of a word”, so “r” does not match inside “user” and “java” does not match inside “javascript”.

Set. A collection with no duplicates. Intersection (∩) = in both; difference (−) = in one but not the other.

What I found by running the matcher (all outputs are real)

Input text	Skills found	What it shows
“C++ and C# developer”	none	**Bug:** the word-boundary check fails after “+” and “#”, so C++ and C# can **never** be matched
“Experienced in C++, Java”	java	Same bug: C++ is dropped, Java is kept
“go to market; excel in teams; r&d work”	excel, go, r	**False positives:** ordinary words match skills (Excel, Go, R)
“express myself; notion of rust and swift and ruby”	express, notion, rust, swift, ruby	More false positives from common words
“Python3 developer”	none	Misses a version suffix
“REST APIs”, “Machine-learning”, “CI-CD”, “scikit learn”	none	Plural, hyphen and spacing variants are missed
“NodeJS, sklearn, ML, PowerBI”	none	Common abbreviations and aliases are missed
“Node.js, scikit-learn, Power BI”	node.js, scikit-learn, power bi	Exact canonical spellings work
“GitHub Actions”	github, github actions	One phrase can count twice (both are in the list)

DON'T SAY

Do not hide these. If an interviewer asks about edge cases, say: *“I know the matcher has false positives and misses; I found them by testing. The fix is a synonym / alias map, lookaround-based boundaries such as (?<!\w)c\+\+(?!\w), and a learned vocabulary.”* Showing you tested your own tool is a strength.

Strengths, gaps and the coverage bar

- **Alphabetical, not by importance.** `sorted(...)[:5]` takes the first five A–Z, so “aws, django, docker, flask, git” appear whether or not they were the key requirements.
- **Capped at 5 each.** A candidate who matches 11 of 13 JD skills shows only 5 strengths.
- **The coverage bar disagrees with the skill score.** The bar uses only the capped lists. Test: a JD with 13 skills and a resume matching 11 gives skill score **84.62**, but strengths 5 + gaps 2 make the bar read **5/7 = 71%**.
- **Extra resume skills earn nothing.** The score only measures coverage of the JD's vocabulary.
- **JD with no known skills.** If the JD mentions none of the 92 keywords (for example a non-technical role) the skill score is 0.0 for every candidate.

The hybrid score and the labels

final = round(0.7 × semantic + 0.3 × skill, 2). Because both inputs are 0–100, so is the final. The table shows the semantic score a candidate needs to reach each label for several skill scores (pure arithmetic from the two formulas).

Skill score	Semantic needed for Moderate (≥ 40)	= raw cosine	Semantic needed for Strong (≥ 65)	= raw cosine
0 (no JD skills known)	57.1	0.629	92.9	0.896
25	46.4	0.548	82.1	0.816
50	35.7	0.468	71.4	0.736
75	25.0	0.388	60.7	0.655
100	14.3	0.307	50.0	0.575

WATCH OUT

Insight worth saying aloud: when the JD has no recognised skills, a candidate needs a raw cosine of about 0.90 to reach “Strong Fit”, which is rarely achievable for a resume against a JD, so almost everyone is labelled Moderate or Not Fit. Even with a perfect skill match the semantic score must be at least 50 (cosine 0.575).

Worked example (illustrative numbers). Resume A: semantic 62.5, skill 75 (6 of 8 JD skills) → 0.7 × 62.5 + 0.3 × 75 = 43.75 + 22.5 = **66.25 → Strong Fit**. Resume B: semantic 45, skill 37.5 (3 of 8) → 31.5 + 11.25 = **42.75 → Moderate Fit**.

Why these thresholds?

The code comment says the labels are “aligned with the assessment spec”, but that specification is not in the repository. Be ready to say where 65 and 40 came from (for example: given by the assessment brief) and that they are not validated against recruiter decisions.

07 The app, parsing and engineering practice

Streamlit in plain English

IN PLAIN ENGLISH

Streamlit runs your Python file from top to bottom *every time the user interacts with a widget*. Widgets return their current value, and output calls draw to the page. That model is simple, but it means anything not stored explicitly is recomputed or lost on the next interaction.

Streamlit call in app.py	What it does here
`st.set_page_config / title / caption`	Page title, wide layout, heading
`st.columns, st.text_area, st.file_uploader(accept_multiple_files=True)`	Two-column input area: job description box and multi-file uploader restricted to PDF and DOCX
`st.button("Analyse Resumes")`	Returns True only on the run triggered by the click; the whole analysis sits inside this if-block
`st.warning / st.error / st.stop`	Show a message and end the script run early
`st.spinner`	“Analysing resumes…” while the loop runs
`st.success / st.expander / st.metric / st.progress`	Top-candidate banner, one collapsible card per candidate, score tiles, coverage bar
`st.download_button`	Sends the pandas DataFrame as CSV
`@st.cache_resource`	Keeps one loaded model instance between reruns (a model is too big to reload each time)

WATCH OUT

A known Streamlit behaviour to be aware of: because results are not saved in `st.session_state` and are only drawn inside the button's if-block, the next widget interaction re-runs the script and the results disappear (many Streamlit versions rerun on download-button clicks too). I did not launch the app, so check how it behaves in your installed version before a live demo. The fix is to store results in `st.session_state`.

Parsing details and risks

- **PDF:** `fitz.open(stream=raw, filetype="pdf")` reads bytes in memory; `page.get_text()` returns each page's text layer; pages are joined with newlines. (The import name `fitz` triggers a deprecation notice in current PyMuPDF; `import pymupdf` is the modern form.)
- **DOCX:** `docx.Document(io.BytesIO(raw))` and `doc.paragraphs`: body paragraphs only. Many resumes put skills or experience in tables, headers or text boxes; those are skipped silently.
- **Error handling:** any exception becomes an empty string; the UI then says the file “could not be read or appears empty”. That is friendly but hides the real cause (corrupt, scanned, password-protected, unsupported).
- **Upload size:** the app sets no limit of its own; Streamlit's default per-file limit applies (200 MB unless configured). Resumes are untrusted files fed to third-party parsers, so keep the libraries updated.
- **Data handling:** files live in memory for the run and are not written to disk by the app. The CSV contains file names, which often include people's names.

Performance and efficiency

Observation	Impact	Better
The model loads when embedder.py is imported (module level), cached by Streamlit	First page load is slow (download then load); later runs reuse it	Keep as is; add a spinner or warm-up message
JD encoded again for each resume	2N model calls instead of N + 1	Encode the JD once; encode resumes as a batch
Resumes processed one after another	Time grows linearly with the number of files; CPU bound	Batch encoding; limit the file count; progress bar
Skill regex compiled each call: 92 patterns × (JD + every resume)	Negligible at this scale, wasteful at scale	Pre-compile one combined pattern; extract JD skills once
The JD and the resume are each scanned for skills twice per resume (in skill_match_score and again in analyze_resume)	Repeated work	Extract once and reuse

Privacy, ethics and fairness (interviewers love this)

IN PLAIN ENGLISH

A resume screener affects people's livelihoods. In many places, automated hiring tools attract scrutiny and sometimes regulation, so showing that you thought about this signals maturity. You do not need legal detail; you do need the principles.

Risk	How it could appear in this app	Mitigation to propose
Biased embeddings	A general pretrained model can pick up social patterns tied to names, schools, gendered wording or locations	Remove names and other personal details before scoring; audit scores across groups; keep a human decision-maker
Formatting penalties	A candidate whose skills sit in a table or a two-column layout loses content (shown in Chapter 4) and scores lower for reasons unrelated to ability	Better parsing; tell recruiters when content may be missing
Vocabulary bias	A fixed technical keyword list favours technical roles and common spellings; non-native phrasing is missed	Aliases; learned vocabulary; role-specific lists
Over-reliance	A single number and a “Not Fit” label can be treated as a verdict	Present as decision support, show reasons, never auto-reject
Privacy	Resumes hold personal data; the model is downloaded from a public hub (the data itself is not uploaded, because encoding happens locally)	Keep processing local, no logging of content, delete exports, follow the employer's data policy

Engineering gaps and how you would close them

Gap in the repo	Why it matters	Fix
No automated tests	Nothing guards the scoring logic; the C++ bug went unnoticed	pytest with a stub model (sample below)
No evaluation data or metrics	No evidence the ranking is good; the weights and thresholds are untested	Small labelled set, Spearman / precision@k, ablations (Chapter 8)
embedder.py imports Streamlit and loads the model at import time	Hard to unit-test and to reuse outside Streamlit	Inject the model; keep Streamlit only in app.py
Results not stored in session_state	Results vanish on the next interaction	Save results in session_state
Requirements have only lower bounds	A future release can break the app	Pin versions (pip freeze) or use a lock file
No logging or detailed errors	Cannot tell why a file failed	Log exceptions; show the reason
README placeholders	Looks unfinished	Real clone URL, screenshots, demo GIF, a license
No Dockerfile or CI	Not reproducible or checked automatically	Dockerfile, GitHub Actions running pytest

A starter test-suite (I ran one like this: all pass; the C++ test is marked as an expected failure)

```
import pytest
from src.analyzer import extract_skills, skill_match_score, get_recommendation
from src.embedder import _normalize_score

@pytest.mark.parametrize("raw,exp", [(0.10,0.0),(0.20,0.0),(0.575,50.0),(0.95,100.0),(1.0,100.0)])
def test_normalize_clamped_and_linear(raw, exp):
    assert _normalize_score(raw) == pytest.approx(exp, abs=0.01)

@pytest.mark.parametrize("score,label", [(39.99,"Not Fit"),(40,"Moderate Fit"),(64.99,"Moderate Fit"),(65,"Stro
ng Fit")])
def test_label_boundaries(score, label):
    assert get_recommendation(score) == label

def test_r_not_inside_user():
    assert "r" not in extract_skills("user interface design")

def test_skill_score_is_share_of_jd_skills():
    assert skill_match_score("python sql", "python sql docker aws") == 50.0

@pytest.mark.xfail(reason="known bug: \\b fails after + and #", strict=True)
def test_cpp_detected():
    assert "c++" in extract_skills("Experienced in C++ and Java")
```

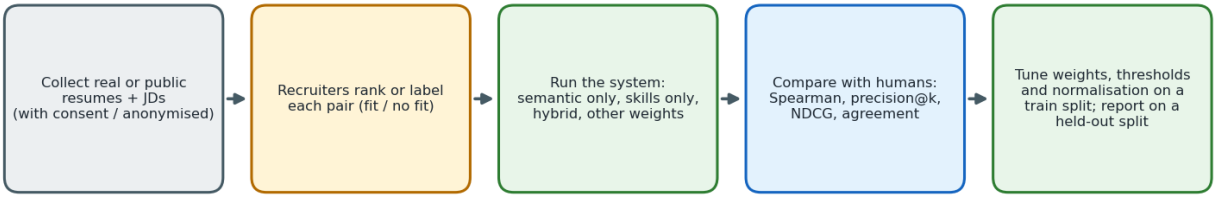
A conftest.py that replaces streamlit and sentence_transformers with tiny fakes lets these run without downloading the model.

08 Evaluation, limitations and the improvement roadmap

What is missing: proof that it ranks well

The repository has no labelled resumes, no metrics and no tests. That is the single biggest question a data-minded interviewer will ask: *"How do you know it works?"* An honest answer plus a clear plan is far stronger than a made-up number.

How I would evaluate the screener (not done in the repository)



Without a labelled set, 70 / 30, the 0.20–0.95 bounds and the 65 / 40 cut-offs are design choices, not measured optima.

Figure 9. A minimal evaluation plan for the screener.

Metric	Plain English	Use here
Spearman rank correlation	How similar two rankings are (−1 to 1)	System ranking vs recruiter ranking for each JD
Precision@k	Of the top k candidates the system picks, how many are truly good	Top-5 / top-10 shortlist quality
Recall@k	Of all truly good candidates, how many are in the top k	Did we lose good people?
NDCG	Ranking quality that rewards putting the best first	Overall order quality
Ablation	Switch parts off to see their effect	Semantic only vs skills only vs hybrid vs a TF-IDF baseline
Weight / threshold tuning	Choose 70 / 30 and 65 / 40 on one split, report on another	Replaces guesses with evidence

Roadmap from quick wins to bigger upgrades

Effort	Improvement	What it fixes
Hours	Lookaround boundaries `(?<!\w)skill(?!\w)`; alias map (nodejs → node.js, sklearn → scikit-learn, ML → machine learning, PowerBI → power bi); plurals and hyphens	C++ / C# misses, synonyms
Hours	Remove ambiguous words (go, r, excel, express, notion, swift, rust, ruby) or require context	False positives
Hours	Encode the JD once; batch-encode resumes; store results in session_state	Speed; vanishing results
Hours	Add a TF-IDF cosine baseline and show it beside the embedding score	Makes the TF-IDF claim true; gives a comparison
Hours	pytest suite with fakes; GitHub Actions; pin dependencies	No tests
1–2 days	Section detection and regexes for years of experience and education (degree keywords); show them in the UI	Makes the original resume claim true
1–2 days	Chunk long texts into windows, embed each, aggregate	256-token truncation
Days	Label a small dataset; tune weights and thresholds; report Spearman and precision@k	No evidence of quality
Days	NER or LLM-based skill extraction; OCR for scanned PDFs	Fixed vocabulary; unreadable files
Weeks	FastAPI back end + React front end, database of jobs and candidates, authentication, bias audit	Production use

SAY IT LIKE THIS

“I treat this as a working prototype. The three things I would do first are to fix the matcher's known false positives and misses, to add a TF-IDF baseline and tests, and to build a small labelled set so I can tune the 70 / 30 weights and the 65 / 40 cut-offs with evidence instead of judgement.”

Honest limitations (what to say before they ask)

- **Fixed vocabulary:** only 92 skills; synonyms and anything new are invisible.
- **Keyword bugs:** C++ and C# never match; common words create false positives.
- **Parsing:** tables, headers and scanned PDFs are lost; multi-column layouts may scramble.
- **Semantic score:** general-purpose model; one vector per document; only the first ~256 word-pieces are read.
- **Unvalidated numbers:** 70 / 30, 0.20–0.95 and 65 / 40 are design choices.
- **No evaluation, tests, authentication or deployment.** No fairness assessment.

09 Glossary: terminology in plain English

Alphabetical. Each row gives a one-breath definition and where it appears in your project.

Term	Plain-English meaning	In your project
Ablation	Turning parts of a system off to measure each part's contribution.	Semantic only vs skills only vs hybrid (proposed).
Bi-encoder	Encodes two texts separately and compares the vectors.	MiniLM encoding resume and JD.
BERT	A transformer language model that reads text in both directions; the base of many NLP models.	MiniLM is a small BERT-style model.
Cache (st.cache_resource)	Keep a computed object so it is not rebuilt each rerun.	The loaded model.
Clamp	Force a value into a range: values below the minimum become the minimum, and likewise for the maximum.	Cosine clipped to 0.20–0.95.
Contrastive learning	Training that pulls related pairs together and pushes unrelated ones apart.	How sentence-embedding models are trained.
Cosine similarity	Cosine of the angle between two vectors; 1 = same direction.	Semantic score.
Cross-encoder	A model that reads two texts together and scores the pair; slower, more accurate.	A possible re-ranker.
CSV	Plain-text table with comma-separated columns.	screening_results.csv.
DataFrame (pandas)	A table object in Python.	Builds the CSV.
Decision support	A tool that informs a human decision rather than making it.	How to describe the screener.
DOCX / python-docx	Word file format / library to read it.	parser._extract_docx.
Embedding	A list of numbers representing the meaning of a text.	384 numbers per text.
Explainability	Being able to say why a result was produced.	Matched and missing skills.
False positive / negative	A wrong match / a missed match.	“excel” the verb; missed “C++”.
Hybrid scoring	Combining several signals into one score.	0.7 semantic + 0.3 skill.
Information extraction / NER	Pulling structured facts (skills, titles, degrees) out of text; NER = named-entity recognition.	Proposed upgrade.
Inverse document frequency	Weight that is high for words appearing in few documents.	Part of TF-IDF (not used).
Keyword matching	Checking whether specific words appear.	Skill half.
Lookaround (regex)	A pattern condition about neighbouring characters without consuming them.	Fix for C++ / C#.
MiniLM	A small distilled transformer family.	all-MiniLM-L6-v2.
Monotonic transform	A change that preserves order.	The 0–100 rescale keeps ranking.
NDCG	Ranking metric that rewards putting the best items first.	Proposed evaluation.
OCR	Optical character recognition: turning images of text into text.	Needed for scanned PDFs.
Parsing	Turning a file into usable text or structure.	parser.py.
PyMuPDF (fitz)	PDF reading library.	PDF text extraction.
Precision@k / Recall@k	Quality / completeness of the top-k results.	Proposed evaluation.
Regular expression	A pattern language for matching text.	`\bskill\b` matching.

Term	Plain-English meaning	In your project
Rescaling / normalisation	Mapping numbers onto a standard range.	Cosine → 0–100.
scikit-learn	Python machine-learning library.	cosine_similarity only.
Semantic similarity	How close two texts are in meaning.	Semantic score.
Sentence Transformers (SBERT)	Library and models that produce sentence embeddings.	SentenceTransformer.
Session state	Streamlit's memory that survives reruns.	Not used yet.
Set (intersection / difference)	Unique collection; ∩ in both, − in one only.	Strengths and gaps.
Sparse vs dense vector	Mostly zeros (TF-IDF) vs all numbers meaningful (embeddings).	Compare in Chapter 5.
Spearman correlation	Agreement between two rankings.	Proposed evaluation.
Streamlit	Python library for quick web apps; reruns the script on interaction.	app.py.
TF-IDF	Word weighting: frequent in this document and rare overall = important.	Not used in this project.
Token / word-piece	A chunk of text (word or sub-word) the model reads.	256-piece limit.
Transformer	Neural architecture using attention; basis of modern NLP.	The model family.
Truncation	Cutting input at a maximum length.	Long resumes are cut.
Vector	An ordered list of numbers.	Embeddings.
Word boundary (\b)	Regex edge between a word character and a non-word character.	Prevents “r” inside “user”.

10 Question bank: more than 50 questions with answers

Chapter 4 already holds the questions tied to your three resume bullets. These cover the rest. Practise saying each answer in 20–40 seconds, then be ready to go deeper.

A. About the project

Q. 1. Walk me through the project end to end.

"A recruiter pastes a job description and uploads resumes in Streamlit. For each file, PyMuPDF or python-docx extracts text. A MiniLM Sentence Transformer embeds the resume and the JD and I take the cosine similarity, rescaled to 0–100. Separately I look for 92 known skills in both texts and compute the share of the JD's skills the resume covers. The final score is 0.7 × semantic + 0.3 × skill, which gives a Strong, Moderate or Not Fit label. Results are sorted, shown with strengths, gaps and an explanation, and can be exported as CSV." (Redraw Figures 1 and 2.)

Q. 2. Why did you build it? Who uses it?

Give your **real** reason (for example a course, a hiring-tech interest, or an assessment brief; the code comments mention an assessment specification). Then: "the user is a recruiter or hiring manager with many resumes per role who wants a fast, explainable first pass." Do not invent a story.

Q. 3. Why hybrid rather than only embeddings or only keywords?

"Embeddings capture meaning but cannot explain themselves; keywords explain themselves but miss paraphrases. The hybrid gives a meaning-aware score plus literal, auditable strengths and gaps."

Q. 4. What was the hardest part, and what are you proudest of?

Choose something real, such as designing the scoring so the output is explainable, or the validation and error handling of messy files. Pair it with an honest lesson: "I later found keyword edge cases such as C++ and I know how to fix them."

Q. 5. How much of this did you write yourself? Did you use AI tools?

Answer **truthfully and specifically**: what you designed and wrote, what came from libraries, tutorials or assistants, and what you checked. The project is only 336 lines, so interviewers can ask you to explain any line. Practise explaining `extract_skills()`, `_normalize_score()` and the hybrid-score line without notes.

Q. 6. Did you train a model?

"No. I used a pretrained Sentence Transformer as is; I did not fine-tune it. The 'AI' part is the pretrained embedding; the skill matching and scoring are rules I wrote."

B. NLP and embeddings

Q. 7. What is an embedding?

"A fixed-length list of numbers that represents the meaning of a text, produced by a neural network so that similar meanings get similar vectors."

Q. 8. What is MiniLM / a Sentence Transformer, and why this model?

"A small distilled transformer fine-tuned so whole sentences map to useful vectors. all-MiniLM-L6-v2 has 6 layers and 384 dimensions, is fast on a CPU and good for general similarity. I did not benchmark others."

Q. 9. What is cosine similarity and why use it?

"The cosine of the angle between two vectors: 1 is the same direction, 0 unrelated. It compares direction, not length, which is how sentence embeddings are meant to be compared."

Q. 10. What does a cosine of 0.6 mean?

"A moderately related pair. It is a relative signal: 0.6 is closer than 0.4, but it is not a 60% probability of anything. That is why I rescale it only for readability."

Q. 11. What does the 0.20–0.95 normalisation do?

"It clamps the raw cosine to that range and stretches it to 0–100, so 0.575 shows as 50. It preserves the order of candidates; its effect is on how the semantic score blends with the skill score and where the labels fall. The bounds are a heuristic."

Q. 12. What happens with a long resume?

"The model truncates at 256 word-pieces, roughly the first 150–200 words, so the semantic score mostly reflects the top of the document. I would chunk the text and aggregate the chunk similarities."

Q. 13. How would you chunk and aggregate?

"Split by sections or sliding windows of about 150 words, embed each, compute similarity to the JD, and take the mean of the top few or the maximum. Better still, compare each JD requirement with the best-matching resume sentence and show that sentence as evidence."

Q. 14. TF-IDF vs embeddings?

"TF-IDF weights words by how frequent they are in a document and how rare across documents; vectors are sparse and only match shared words. Embeddings are dense and capture meaning across different words. TF-IDF is a great explainable baseline. I did not use TF-IDF in this project."

Q. 15. Bi-encoder vs cross-encoder?

"A bi-encoder embeds each text separately, so you can precompute and compare quickly; I use that. A cross-encoder reads the pair together and is more accurate but slower, so it is usually used to re-rank a shortlist."

Q. 16. Could you use an LLM instead?

"Yes, it can read a whole resume and explain its reasoning, but it is slower and costlier, can be inconsistent between runs, and raises privacy and bias concerns because resumes go to a model. I would consider it for explanations on a shortlist, with a human reviewing."

Q. 17. How would you fine-tune the embedding model for resumes?

"Collect resume and JD pairs labelled good or bad match, then fine-tune with sentence-transformers using a pair or contrastive loss, holding out a test set. It needs labelled data, which I do not have yet."

Q. 18. Does the model understand negation or seniority?

"Only weakly. 'No experience with Python' and 'experience with Python' share most words and can embed close together, and seniority is not modelled. That is why the skill half and a human reviewer matter, and why I would add experience extraction."

Q. 19. What is a transformer, in one minute?

"A neural network that uses attention to weigh how much each word should look at every other word, so each word's representation depends on its context. BERT-style models use it to read text in both directions."

Q. 20. Does it work for other languages?

"This model is English-focused. A multilingual sentence-embedding model would be needed for other languages, and the keyword list is English too."

C. Scoring

Q. 21. Why 70 / 30 and how would you choose the weights?

"It was a design choice. To choose properly, I would label a set of resume–JD pairs, then compare rankings for several weights with Spearman and precision@k on a held-out set."

Q. 22. What does a final score of 65 mean and where does it come from?

"It is the cut-off for 'Strong Fit' in my blended scale, which the code says follows the assessment specification. It is not a hiring probability and is not validated against recruiter decisions."

Q. 23. What if the job description names none of your 92 skills?

"The skill score becomes 0 for everyone, so the final is at most 70 and a candidate needs a raw cosine of about 0.90 to reach Strong Fit. Almost everyone would be Moderate or Not Fit, so I would warn the user or fall back to semantic-only labels."

Q. 24. How are strengths and gaps chosen?

"Strengths are skills in both JD and resume; gaps are JD skills missing from the resume. Each is sorted A–Z and cut to five, so they are not ranked by importance. I would rank by how often or how prominently the JD mentions each skill."

Q. 25. Why does the coverage bar sometimes differ from the skill score?

"The bar uses the capped strengths and gaps lists (max 5 each), while the score uses the full sets. With 13 JD skills and 11 matched, the score is 84.6 but the bar shows 5 of 7. I would compute the bar from the full sets."

Q. 26. Could someone game the system?

"Yes. Listing every JD skill raises the skill score, and copying JD phrases raises similarity. Repeating a keyword does not help the skill score because it counts presence, but stuffing the list does. Mitigations are context checks, experience verification, and a human review."

Q. 27. Is the score stable? Would the same resume score the same twice?

"Yes. The model is deterministic and the rules are deterministic, so the same inputs give the same score. An LLM-based scorer would not guarantee that."

Q. 28. Why does the explanation sentence say 'shows proficiency'?

"It is a template filled with matched skills, so 'proficiency' overstates it: the resume only *mentions* the skill. I would soften it to 'mentions'."

D. Parsing and skill extraction

Q. 29. How does skill extraction work?

"Lower-case the text, collapse whitespace, then test 92 keywords with a word-boundary regex and collect the matches in a set. Boundaries stop 'r' matching inside 'user'."

Q. 30. What edge cases do you know about?

"I tested it: C++ and C# never match because the boundary check fails after '+' and '#'; ordinary words like 'excel', 'go' and 'express' create false positives; aliases such as NodeJS, sklearn and ML are missed. A lookaround pattern and an alias map would fix most of it."

Q. 31. How would you improve skill extraction beyond a list?

"Use an alias map and fuzzy matching first, then learn terms from a corpus of job descriptions, or use an NER model or an LLM to extract skills with context such as 'used Spark for three years'."

Q. 32. How would you extract years of experience and education?

"Detect sections by headings, parse date ranges such as '2021 – 2024' and sum non-overlapping months, and match degree keywords (B.E., B.Tech, M.Sc., Ph.D.) in the education section. Or use an NER or LLM extractor and validate it on a labelled sample."

Q. 33. What about scanned PDFs, tables and two-column resumes?

"Scanned PDFs have no text layer: OCR such as Tesseract is needed. DOCX tables and headers are skipped by python-docx's paragraph list, so I would also read table cells and headers. Two-column PDFs need layout-aware extraction because plain text extraction can interleave columns."

Q. 34. Why PyMuPDF and python-docx?

"PyMuPDF is fast and handles in-memory bytes well for PDFs; python-docx is the standard library for DOCX. Both work directly on the uploaded bytes without saving files."

E. Streamlit and engineering

Q. 35. How does Streamlit run your code?

"It re-runs the script top to bottom on every widget interaction. That is why I cache the model with st.cache_resource, and why results should be stored in st.session_state, which I have not done yet."

Q. 36. Why cache the model?

"Loading the model is slow and memory-heavy; the cache keeps one instance across reruns and sessions."

Q. 37. How would you test it?

"Unit tests for the normalisation bounds, label boundaries, skill regex cases and hybrid arithmetic, with a stub model so no download is needed, plus parser tests with generated PDF and DOCX files, and a GitHub Actions workflow to run them."

Q. 38. How would you deploy it?

"Streamlit Community Cloud for a quick demo, or a Docker image on a small VM. I would pin dependencies, pre-download the model into the image, and add authentication since resumes are personal data."

Q. 39. How would it scale to thousands of resumes?

"Parse and embed in batches, precompute and store embeddings in a vector index such as FAISS or a vector database, encode each JD once, and rank by vector search followed by a re-ranker. Move the logic into an API service with a queue."

Q. 40. How do you protect candidate privacy?

"Everything is processed in memory and nothing is stored; the model runs locally. For production I would add access control, avoid logging resume text, anonymise names before scoring, delete exports and follow the employer's data policy."

Q. 41. What would you monitor?

"Parse-failure rate, share of empty-text files, score distributions, the share of JDs with no recognised skills, latency per resume, and recruiter overrides."

Q. 42. Why is unpinned requirements.txt a risk?

"A new major release of Streamlit, sentence-transformers or PyMuPDF could change behaviour or break the app. Pin versions or use a lock file."

F. Evaluation, fairness and ethics

Q. 43. How do you know it works?

"I cannot claim that yet: there is no labelled evaluation. I can show the logic and my tests of its behaviour. The plan is a small labelled set and metrics such as Spearman and precision@k, with ablations of semantic-only, skills-only and hybrid."

Q. 44. How would you check it for bias?

"Score matched resumes that differ only in a name, gender-coded wording or school and compare the scores; compare selection rates and score distributions across groups; remove personal details before scoring; and keep a human decision-maker."

Q. 45. Is automated screening allowed?

"It depends on the jurisdiction and the employer's policy, and some places regulate automated employment tools. I would keep it as decision support with human review and involve the employer's legal and HR teams."

Q. 46. Why might a recruiter distrust the score?

"Because a single number hides the reasoning. That is why I show matched and missing skills and an explanation, and why the label should never trigger an automatic rejection."

Q. 47. What if it rejects a great candidate?

"A false rejection is the costly error. Use the tool to prioritise and surface candidates, not to eliminate, sample the low-scoring resumes for review, and track recruiter overrides."

Q. 48. Could the model have learned bias from its training data?

"Yes. Pretrained language models reflect patterns in their training text. I have not audited this model for hiring bias, so I would not rely on it alone."

G. Data-engineering angle and reflection

Q. 49. How is this relevant to data engineering?

"It is a small pipeline over unstructured data: ingest files, parse and clean text, derive features (embeddings and skill sets), score, and export. It involves data-quality questions such as unreadable files and missing content, and batch processing choices."

Q. 50. How would you turn it into a proper pipeline?

"Land files in object storage, parse to text with failure logging, run data-quality checks (empty text, length, language), embed in batches, store vectors and extracted skills in a database, compute scores on demand, and orchestrate with a scheduler, tracking parse and score metrics."

Q. 51. What would you do differently?

"Start with tests and a small labelled dataset, build the matcher with aliases and lookaround boundaries, read tables in DOCX, and encode the JD once."

Q. 52. What is the biggest weakness?

Use the honest limitations statement in Chapter 11.

Q. 53. Show me where in the code X happens.

Be able to open, without searching: `embedder._normalize_score` · `embedder.compute_similarity` · `analyzer.extract_skills` · `analyzer.skill_match_score` · `analyzer.get_recommendation` · `parser.extract_text` · `app.py → hybrid score line and CSV export`

11 Trap questions and honest limitations

These test whether you overclaim. A precise limitation is stronger than an exaggerated claim.

Trap	Honest answer
“Your resume says it extracts experience and education.”	“That wording was too broad: it extracts skills only, against a 92-keyword list. Experience and education extraction is a planned improvement.” (Fix the resume so this never arises.)
“Where did you use TF-IDF?”	“Not in this project. I know it and use it as a comparison, but the skills line should not have listed it for this project.”
“What is the accuracy?”	“I have no labelled data, so I cannot give an accuracy. I can show how I would evaluate it: labelled pairs, Spearman and precision@k, and ablations.”
“Does semantic matching actually beat keyword search?”	“That is the design intent and it is plausible, but I have not measured it. A TF-IDF or keyword-only baseline in an ablation would show it.”
“Why 70 / 30?”	“A design choice to let meaning dominate while keeping an explainable keyword component. I did not tune it.”
“Where do 0.20 and 0.95 come from?”	“Heuristic bounds so scores read naturally. They preserve ranking and are not statistically derived.”
“Is a score of 80 an 80% chance of being a good hire?”	“No. It is a position on my blended scale, not a probability.”
“Does it handle C++ and C#?”	“No: I found that the word-boundary regex never matches them. The fix is a lookaround pattern; I would add a test for it.”
“Does the model read the whole resume?”	“Only the first ~256 word-pieces, about 150–200 words. Chunking would fix this.”
“Can it read scanned PDFs, tables or headers?”	“No. Scanned PDFs need OCR, and DOCX tables and headers are skipped by the paragraph-only reader.”
“Did you train or fine-tune the model?”	“No, I used the pretrained all-MiniLM-L6-v2 as is.”
“Is it biased?”	“I have not measured it. Pretrained models and keyword lists can encode bias; I would audit with counterfactual tests, remove personal details and keep human review.”
“Do you store resumes?”	“No: they are processed in memory; the only output is the CSV the user downloads, which contains file names.”
“Is it production-ready?”	“No: it is a prototype with no tests, authentication or deployment, as the README's limitations indicate.”

The honest limitation statement (memorise this one)

> **SAY IT LIKE THIS**
> “It is a transparent prototype. The semantic score comes from a pretrained general-purpose model that only reads the first few hundred words of each document; the skill score comes from a fixed 92-keyword list with known false positives and misses; and the 70 / 30 weights, the score rescaling and the 65 / 40 labels are design choices I have not validated against recruiter judgements. I would use it as decision support with a human reviewer, and my next steps are tests, a small labelled evaluation and better skill and experience extraction.”

If you do not know the answer

- Say what you know, then reason: “I haven't measured that; I would test it by …”.
- Offer to open the code: “analyzer.py line … shows exactly that.”
- Never invent a number: this project has no accuracy figures, and that is fine if you say so clearly.

12 One-page cheat sheet

Fact	Value
Code size	336 lines in 4 files: app.py 160 · analyzer.py 94 · embedder.py 45 · parser.py 37
Stack	Python · Streamlit · sentence-transformers · scikit-learn (cosine only) · PyMuPDF · python-docx · pandas
Model	all-MiniLM-L6-v2 · 384-dim · 6 layers · ~256 word-piece input limit · pretrained, not fine-tuned
Semantic score	(clamp(cosine, 0.20, 0.95) − 0.20) ÷ 0.75 × 100
Skill score	matched ÷ JD skills × 100 (0 if the JD has no known skills)
Final	0.7 × semantic + 0.3 × skill (rounded to 2 dp)
Labels	≥ 65 Strong Fit · ≥ 40 Moderate Fit · below 40 Not Fit
Vocabulary	92 skills, 9 groups: languages 14 · data 14 · ML/AI 13 · cloud 11 · web 10 · backend 9 · tools 8 · soft 7 · viz 6
Strengths / gaps	Sets; sorted A–Z; first 5 each
Output	Ranked cards with 3 scores, strengths, gaps, label, explanation; CSV with Rank and 8 columns
Parsing	PDF text layer via PyMuPDF; DOCX paragraphs only (tables / headers lost); scanned PDF, .doc, corrupt → “could not be read”
Known bugs (tested)	C++ and C# never match · false positives on excel, go, r, express, notion, rust, swift, ruby · aliases missed · coverage bar uses capped lists
Strong Fit needs	Skill 100 → semantic ≥ 50 · skill 50 → ≥ 71.4 · skill 0 → ≥ 92.9
Not in the repo	Tests · evaluation · metrics · experience / education extraction · TF-IDF · auth · deployment

Say	Never say
“Decision support”, “hybrid: semantic for meaning, keywords for explainability”, “92-keyword vocabulary”, “pretrained model, not fine-tuned”, “design choices I have not tuned”	“Accurate”, “extracts experience and education”, “uses TF-IDF”, “unbiased”, “production-ready”, “trained my own model”, any accuracy percentage

Before the interview

- Resume and README corrected (Chapter 1).
- Ran the app locally with three fictional resumes and a JD; know how results behave after clicking download.
- Redrawn Figures 1 and 2 from memory; written the hybrid formula and thresholds without notes.
- Said the 30-second and 90-second pitches and the honest-limitation statement aloud.
- Opened analyzer.py and embedder.py and explained each function without notes.

13 Appendix A: run it and demo it in two minutes

Run locally (from the README)

```
git clone <your repository URL>
cd <repository folder>
python -m venv venv
venv\Scripts\activate     (Windows)   |   source venv/bin/activate (Mac / Linux)
pip install -r requirements.txt
streamlit run app.py
```

On the first run the model is downloaded from Hugging Face, which can take a while; do this before the interview, on a good connection. Streamlit prints a local URL (usually http://localhost:8501).

Prepare demo files

- Write a short, realistic **job description** (for example for a Data Engineer: Python, SQL, PySpark, ETL, Docker, AWS).
- Create 3 **fictional** resumes: a strong match, a weak match and a paraphrase-heavy one (same skills in different words). Never demo with a real person's resume.
- Include one scanned or unreadable file so you can show the error message, and one DOCX with a skills table if you want to show the limitation honestly.

Two-minute demo script

Step	Show	Say
1	Paste the JD; upload the three resumes; click Analyse	“It reads each file, computes two scores and blends them 70 / 30.”
2	Open the top candidate's card	“Semantic, skill and final scores, strengths, gaps and a plain-English insight. Strengths and gaps are literal matches, so a recruiter can verify them.”
3	Compare the paraphrase-heavy resume's semantic and skill scores	“Semantic stays reasonable even when keywords differ, which is why the hybrid helps.”
4	Upload the unreadable file	“It reports the file and carries on with the others.”
5	Download the CSV	“Rank, scores, strengths, gaps, label and explanation for sharing.”
6	Open analyzer.py and point at SKILLS and the regex	“Matching is intentionally simple and auditable; here are its limits and how I would fix them.”

14 Appendix B: where things are in the repository

File	Be able to say
README.md	Overview, features, tech stack, how it works (70 / 30), setup, limitations and future work. **Replace the placeholder clone URL.**
requirements.txt	streamlit, sentence-transformers, scikit-learn, pymupdf, python-docx, pandas; lower bounds only.
app.py	Page layout, input checks, per-file loop, hybrid score line, explanation template, sort, results UI, CSV export.
src/parser.py	extract_text(file) with _extract_pdf and _extract_docx; returns an empty string on any failure.
src/embedder.py	load_model (cached), _normalize_score (0.20–0.95 → 0–100), compute_similarity (validation, encode, cosine).
src/analyzer.py	SKILLS (92), normalize, extract_skills (regex), analyze_resume (strengths / gaps), skill_match_score, get_recommendation.
.gitignore	venv, __pycache__, *.pyc, .env